

Informe: Función Hash de Una Sola Vía

Seguridad Informática

Luis Fernando Lopez Pardo - 20241020045

Sofía Rivera - 20242020224

Juan Esteban Moreno Durán - 20232020107

Juan Esteban Avila Trujillo - 20251020054

25 de junio de 2026

Índice

1. Introducción	2
2. Funcionamiento General del Algoritmo	2
3. Fórmulas Utilizadas	2
3.1. Reducción del código ASCII (pérdida de información)	2
3.2. Actualización del estado interno	3
3.3. Rotación de bits	3
3.4. Mezcla adicional (finalización por carácter)	3
4. ¿Por Qué es una Función de Una Sola Vía?	4
5. Uso de la Sal (<i>Salt</i>)	4
6. Generador de Números Pseudoaleatorios	5
6.1. ¿Por qué no se integró dentro de la función hash?	5
7. ¿Cada Hash Generado es Único?	5
7.1. Consistencia	5
7.2. Unicidad y colisiones	5
8. Conclusiones	6

1. Introducción

En este informe se documenta el diseño e implementación de una función hash de una sola vía. El objetivo principal fue construir un algoritmo que transforme una palabra en un código numérico de forma que sea sencillo obtener el hash a partir de la entrada, pero prácticamente imposible recuperar la entrada original a partir del hash.

Para cumplir este objetivo se incorporó una etapa deliberada de **pérdida de información** dentro del proceso de generación del hash, lo cual es la clave que le da el carácter irreversible al algoritmo.

Adicionalmente, se desarrolló por separado un generador de números pseudoaleatorios basado en una variante del método congruencial lineal (LCG). En este informe también se justifica la razón por la cual dicho generador **no fue integrado** dentro de la función hash.

2. Funcionamiento General del Algoritmo

El algoritmo recibe dos entradas: una **palabra** ingresada por el usuario y una **sal** (*salt*) fija definida internamente. Ambas se concatenan para formar la cadena que será procesada.

Los pasos generales del algoritmo son:

1. Concatenar la palabra con la sal: $\text{Entrada} = \text{Palabra} + \text{Salt}$.
2. Obtener el código ASCII de cada carácter de la cadena.
3. Reducir cada código ASCII sumando sus dígitos (pérdida de información).
4. Mezclar el resultado con un estado interno acumulado.
5. Aplicar rotación de bits y operación XOR para dispersar los valores.
6. Producir el hash final a partir del estado resultante.

3. Fórmulas Utilizadas

3.1. Reducción del código ASCII (pérdida de información)

Cada carácter de la cadena se convierte primero a su valor ASCII y luego se reemplaza por la **suma de sus dígitos decimales**. La fórmula general es:

$$S(A) = d_1 + d_2 + \cdots + d_n$$

donde A es el valor ASCII del carácter y $d_1, d_2, \dots, d_n$ son los dígitos que lo conforman.

Ejemplo:

$$'A' \rightarrow A = 65 \rightarrow 6 + 5 = 11$$

$$'8' \rightarrow A = 56 \rightarrow 5 + 6 = 11$$

$$'J' \rightarrow A = 74 \rightarrow 7 + 4 = 11$$

Los tres caracteres distintos producen el mismo valor reducido (11), lo que confirma la pérdida irreversible de información.

3.2. Actualización del estado interno

Después de obtener $S(A)$ para cada carácter, el estado interno se actualiza mediante una multiplicación seguida de una operación XOR:

$$\text{Estado} = (\text{Estado} \times 131) \oplus S(A)$$

donde:

- 131 es una constante prime elegida para dispersar los bits del estado.
- $\oplus$ denota la operación XOR a nivel de bits.

3.3. Rotación de bits

Tras la actualización, se realiza una rotación circular de 5 posiciones hacia la izquierda sobre el estado:

$$\text{Estado} = \text{RotL}(\text{Estado}, 5)$$

Esta operación redistribuye los bits del estado, haciendo que cualquier pequeño cambio en la entrada genere diferencias amplias en el hash final (efecto avalancha).

3.4. Mezcla adicional (finalización por carácter)

Al terminar cada carácter se aplica una última mezcla con un desplazamiento de bits:

$$\text{Estado} = \text{Estado} \oplus (\text{Estado} \gg 7)$$

El desplazamiento a la derecha de 7 posiciones ($\gg 7$) y la posterior operación XOR garantizan que los bits de mayor peso también influyan en los bits de menor peso del estado.

4. ¿Por Qué es una Función de Una Sola Vía?

La propiedad de irreversibilidad del algoritmo se fundamenta principalmente en la **pérdida de información** generada en la etapa de reducción de dígitos.

Como se mostró anteriormente, múltiples valores ASCII distintos pueden producir el mismo valor reducido:

$$65 \rightarrow 11, \quad 56 \rightarrow 11, \quad 74 \rightarrow 11$$

Si solo se conoce el valor 11, es **imposible determinar** con certeza cuál era el código ASCII original, ya que varios candidatos son igualmente válidos. Esta ambigüedad se propaga a través de todas las demás operaciones del algoritmo.

Adicionalmente, las operaciones XOR, la rotación de bits y el desplazamiento mezclan la información de todos los caracteres de forma que el estado final depende de la totalidad de la entrada de manera entrelazada. Esto significa que:

- No existe una operación inversa directa para *deshacer* el XOR con un estado desconocido.
- La rotación de bits no es invertible sin conocer el estado exacto en ese punto del proceso.
- El encadenamiento de operaciones hace que un pequeño cambio en la entrada modifique drásticamente el hash final.

Por todas estas razones, el algoritmo cumple con la característica fundamental de una función hash de una sola vía: es computacionalmente inviable recuperar la entrada a partir del hash.

5. Uso de la Sal (*Salt*)

Además de la palabra ingresada por el usuario, el algoritmo incorpora una **sal** fija. La entrada real que procesa el algoritmo es:

$$\text{Entrada} = \text{Palabra} + \text{Salt}$$

Por ejemplo, si la palabra es HOLA y la sal es Z9xY:

$$\text{Entrada} = \text{HOLA} + \text{Z9xY} = \text{HOLAZ9xY}$$

La sal cumple dos funciones importantes:

- Hace que el hash dependa no solo de la palabra sino también de un valor adicional controlado por el diseñador del sistema.
- Dificulta los ataques de diccionario precalculados (*rainbow tables*), ya que un atacante necesitaría conocer la sal para intentar reproducir el hash.

6. Generador de Números Pseudoaleatorios

6.1. ¿Por qué no se integró dentro de la función hash?

Se tomó la decisión de **no incorporar** un generador de números pseudoaleatorios dentro de la función hash. La razón principal es la siguiente:

Una función hash debe ser determinista: la misma entrada siempre debe producir exactamente el mismo hash.

Si se incorporaran números pseudoaleatorios directamente en el proceso (por ejemplo, para modificar el estado en cada ejecución), el resultado cambiaría entre distintas llamadas al algoritmo, incluso para la misma palabra de entrada. Esto haría imposible verificar hashes, ya que:

Hash(HOLA) = 12345 (primera ejecución)

Hash(HOLA) = 67890 (segunda ejecución)

Esto inutilizaría el propósito del hash, que es comparar valores de forma reproducible.

Dicho esto, el generador pseudoaleatorio *sí podría* haberse utilizado para generar la sal de forma dinámica **antes** de invocar la función hash, o para derivar una clave de sesión. En esos contextos, la aleatoriedad es deseable. Sin embargo, dentro del núcleo del algoritmo hash, rompe la propiedad de determinismo y por eso se descartó.

7. ¿Cada Hash Generado es Único?

7.1. Consistencia

El algoritmo es completamente determinista: la misma entrada siempre produce el mismo resultado.

Hash(HOLA) = 12345 (todas las ejecuciones)

7.2. Unicidad y colisiones

Sin embargo, que el resultado sea consistente no implica que sea único para todas las entradas posibles. El fenómeno en el que **dos entradas distintas producen el mismo hash** se denomina **colisión**.

Las colisiones son inevitables en cualquier función hash por una razón matemática fundamental: el dominio de entrada (todas las palabras posibles) es infinito, mientras que el rango de salida (los hashes posibles) es finito. Por el *Principio del Palomar*, necesariamente deben existir colisiones.

En el caso particular de este algoritmo, la probabilidad de colisión es **mayor que en funciones hash criptográficas estándar** (como SHA-256) debido a la etapa de suma de dígitos, que agrupa muchos valores ASCII distintos bajo un mismo resultado. Esto incrementa las chances de que la cadena de operaciones subsiguientes produzca estados idénticos.

En resumen:

- **Consistencia garantizada:** la misma entrada siempre produce el mismo hash.
- **Unicidad no garantizada:** entradas distintas pueden producir el mismo hash (colisión).
- **La pérdida de información** en la suma de dígitos aumenta la probabilidad de colisión respecto a algoritmos que preservan toda la información de cada carácter.

8. Conclusiones

Se diseñó e implementó una función hash de una sola vía que transforma una palabra en un código numérico mediante una cadena de operaciones de reducción, mezcla y dispersión de bits.

Los aspectos más relevantes del trabajo son los siguientes:

1. **Irreversibilidad:** la propiedad de una sola vía se logra principalmente gracias a la pérdida de información que ocurre al reemplazar cada código ASCII por la suma de sus dígitos. Múltiples entradas posibles se mapean al mismo valor reducido, haciendo imposible la inversión exacta.
2. **Uso de sal:** la incorporación de una sal fija fortalece el algoritmo al hacer que el hash dependa de un valor adicional desconocido para un potencial atacante.
3. **Exclusión del generador pseudoaleatorio:** el generador LCG modificado desarrollado en paralelo no fue integrado en la función hash para preservar el determinismo, que es una propiedad fundamental e indispensable de cualquier función hash.
4. **Colisiones:** aunque el algoritmo es consistente, no puede garantizar la unicidad absoluta de los hashes generados. La posibilidad de colisiones es inherente al diseño y se ve aumentada por la etapa de pérdida de información.